



ШПАРГАЛКА QA

ОСНОВЫ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Автор: Евгений Гусинец

ТГ-канал: QA  4Life



БЛОК 1

ЧТО ТАКОЕ ТЕСТИРОВАНИЕ?

1

Определение · Необходимость · Парадигмы · Навыки · Роли QA/QC

СОДЕРЖАНИЕ БЛОКА 1

- 1.1** **Определение тестирования** ●
 Официальное определение ISTQB · ключевые слова · Static vs Dynamic
- 1.2** **Почему тестирование необходимо** ●
 Природа ошибок ПО · стоимость дефекта $\times 1 \rightarrow \times 1000$ · катастрофы · Shift-Left
- 1.3** **Тестирование $\neq$ поиск ошибок** ●
 Старая vs. новая парадигма · Positive / Negative Testing · Exhaustive Testing
- 1.4** **Как перейти к тестированию** ●
 Навыки QA 2025–2026 · карта Junior / Middle / Senior · инструменты
- 1.5** **Разница: QA / QC / Testing** ●
 Иерархия Testing $\subset$ QC $\subset$ QA · сравнение по 7 критериям · обязанности ролей

"Тестирование — это не фаза. Это образ мышления."

— Принцип современного QA

ОПРЕДЕЛЕНИЕ (ISTQB / ГОСТ Р 56920)

Тестирование (Testing) — процесс, охватывающий все активности жизненного цикла ПО (как статические, так и динамические), связанные с:

- планированием тест-активностей
- подготовкой тестовых условий и тест-кейсов
- выполнением тестов
- оценкой программного продукта и сопутствующих артефактов

Цели тестирования:

- 1 Проверка соответствия продукта заявленным требованиям
- 2 Демонстрация пригодности продукта для использования по назначению
- 3 Обнаружение дефектов

СВОИМИ СЛОВАМИ

Тестирование — это систематическая проверка того, что программный продукт работает так, как ожидается, и выявление мест, где он работает иначе.

СТАТИЧЕСКОЕ

Без запуска кода.
Просмотр требований, документации, кода (code review, инспекции, walkthroughs).

ДИНАМИЧЕСКОЕ

С запуском кода.
Выполнение тест-кейсов, проверка поведения системы в рантайме.

КЛЮЧЕВЫЕ СЛОВА §1.1

Статическое

Динамическое

Оценка

Дефекты

Требования

⚠ Тестирование ≠ Отладка (Debugging)

Тестирование — обнаруживает симптомы (что не работает). Отладка (Debugging) — находит и устраняет первопричину дефекта. Это разные активности, которые выполняют разные роли.

ГЛОССАРИЙ §1.1**Ошибка человека (Error)**

Действие человека, приводящее к неправильному результату. Разработчик неверно понял требование или допустил опечатку в коде.

Дефект / Баг (Defect / Bug / Fault)

Исъян в компоненте или системе, способный привести к сбою. Результат ошибки человека, зафиксированный в артефакте (коде, документе).

Отказ системы (Failure)

Отклонение поведения компонента или системы от ожидаемого. Проявляется в рантайме при выполнении дефектного кода.

Тест-кейс (Test Case)

Набор предусловий, входных данных, действий, ожидаемых результатов и постусловий, разработанный для конкретной тест-цели.

Тест-основа (Test Basis)

Совокупность документов (требования, дизайн, код, стандарты), на основе которых разрабатываются тест-кейсы.

Тест-оракул (Test Oracle)

Источник для определения ожидаемого результата теста: спецификация, предыдущая версия системы, экспертное суждение.

ЦЕПОЧКА ДЕФЕКТОВ

1.2 ПОЧЕМУ ТЕСТИРОВАНИЕ НЕОБХОДИМО

Природа ошибок · Стоимость дефекта · Реальные катастрофы · Shift-Left

ОСНОВЫ ТЕСТИРОВАНИЯ ПО

Программное обеспечение создаётся людьми → люди ошибаются по природе своей.
Ошибка человека (Error) → дефект в коде (Defect) → сбой системы в рантайме (Failure).
Именно поэтому тестирование → обязательная часть разработки, а не опциональная активность.

СТОИМОСТЬ ИСПРАВЛЕНИЯ ДЕФЕКТА

Чем позже обнаружен дефект — тем дороже его исправление

Требования (Requirements)		× 1
Дизайн (Design)		× 5
Разработка (Development)		× 10
Тестирование (Testing)		× 20
Продакшн (Production)		× 100 — 1000

Исправить дешевле всего

Ещё можно исправить до релиза

Критично: пользователи уже видят баг

 **Вывод:** Найти дефект на этапе требований обходится в 100–1000 раз дешевле, чем в продакшне. Принцип Shift-Left Testing — тестируй раньше.

КАТАСТРОФЫ ИЗ-ЗА ОШИБОК ПО

Реальные случаи из истории — цена программных ошибок



Ракета Ariane 5 (1996)

Переполнение 64-битного числа при записи в 16-битную переменную. Система аварийной защиты уничтожила ракету через 37 секунд после старта.



Ущерб: \$370 млн



04.06.1996



Mars Climate Orbiter (1999)

Команда Lockheed Martin передавала данные тяги в имперских единицах (фунт-сила-с), команда NASA ожидала метрические (Н-с). Зонд вышел на слишком низкую орбиту и сгорел.



Ущерб: \$327 млн



Сентябрь 1999



Therac-25 (1985–1987)

Ошибка синхронизации в медицинском радиационном ускорителе. Race condition при быстром вводе оператора. Пациенты получили смертельные дозы радиации в 100× норму.



Погибших: 3 человека, тяжело пострадавших: 3

В медицине, авиации и финансах программные ошибки = угроза жизни людей. Тестирование — это не формальность.

▶ ПРИНЦИП СДВИГА ВЛЕВО — SHIFT-LEFT TESTING



БЕЗ Shift-Left:

Баги попадают в продакшн · Пользователи находят дефекты · Репутационные потери · Исправление стоит ×100–1000

С Shift-Left:

Тесты пишутся параллельно с требованиями · QA участвует с первого дня · Дефекты дешёвы · Качество встроено в процесс

Positive Testing (Test to Pass) · Negative Testing (Test to Fail) · Покрывшие сценариев

☑ POSITIVE TESTING

Test to Pass · «Счастливый путь»

**ПОЗИТИВНОЕ ТЕСТИРОВАНИЕ**

Проверка того, что система **КОРРЕКТНО РАБОТАЕТ** при валидных входных данных и ожидаемых условиях использования. Имитируем поведение обычного пользователя.

Вопрос: "Работает ли система как задумано?"

- Входные данные — валидные, в допустимом диапазоне
- Условия — штатные, предусмотренные требованиями
- Ожидание — система выполняет действие успешно

💡 "Happy Path" — сценарий довольного пользователя

✗ NEGATIVE TESTING

Test to Fail · «Пути ошибок»

**НЕГАТИВНОЕ ТЕСТИРОВАНИЕ**

Проверка того, что система **КОРРЕКТНО ОБРАБАТЫВАЕТ** невалидные данные, ошибки и граничные условия. Имитируем поведение злоумышленника, неопытного пользователя или сбоя среды.

Вопрос: "Не сломается ли система при неправильном вводе?"

- Входные данные — невалидные, пустые, выходящие за границы
- Условия — нештатные, ошибочные, экстремальные
- Ожидание — система отображает корректное сообщение об ошибке

💡 "Sad Path / Error Path" — сценарий ошибки или злоупотребления

ПРИМЕРЫ ДЛЯ ТИПИЧНЫХ СЦЕНАРИЕВ QA

Поле формы регистрации: email, пароль, возраст. REST API: POST /register

Сценарий	☑ Positive (ожидаемый Pass)	✗ Negative (ожидаемый Fail)
Email	user@example.com → 201 Created, письмо отправлено	user@example (без домена), пустая строка "", user@, @@example.com → 400 Bad Request + сообщение об ошибке
Пароль	P@ssw0rd123 (8 символов, буквы+цифра+спецсимвол) → 201 Created	"123" (слишком короткий), "" (пустой), "password" (без спецсимволов) → 422 Validation Error
Возраст	25 (допустимый диапазон 18–99) → поле принято	17 (меньше минимума), 150 (больше максимума), -1 (отрицательное), "двадцать пять" (строка вместо числа) → ошибка валидации
Загрузка файла	photo.jpg, 2 MB, JPEG — успешная загрузка, превью отображается	exploit.exe (неверный формат), 50 MB (превышение лимита), пустой файл 0 KB → соответствующее сообщение об ошибке
Авторизация	Верный логин + верный пароль → 200 OK, токен выдан	Верный логин + неверный пароль → 401 Unauthorized. Несуществующий логин → 404 или 401. 6-я попытка подряд → блокировка аккаунта
Поиск	"iPhone 15" → список товаров, сортировка работает	"" (пустой запрос) → подсказка или ошибка, не краш. SQL: "'; DROP TABLE--" → экранируется, XSS: "<script>alert(1)</script>" → отображается как текст
Числовое поле	Количество: 3 → товар добавлен 3 раза	0 (ноль), -5 (отрицательное), 9999999 (overflow), 1.5 (дробное для целочисленного поля), буква "к" → валидация отклоняет
API: токен авторизации	Bearer eyJhbGc... (валидный JWT) → 200 OK, данные возвращены	Без заголовка Authorization → 401. Истёкший токен → 401. Случайная строка вместо JWT → 401 или 400

КОГДА И КАК ПРИМЕНЯТЬ**📋 НАЧНИ С POSITIVE**

Всегда сначала убедись, что "счастливый путь" работает. Если основной сценарий сломан — negative тестировать бессмысленно.

⚡ ДОБАВЛЯЙ NEGATIVE

После подтверждения позитивных сценариев переходи к граничным значениям, пустым полям, неверным форматам и нагрузочным сценариям.

⚖️ БАЛАНС 70/30

Типичное соотношение в проекте: 70% позитивных тест-кейсов + 30% негативных. При высоких требованиях безопасности — 50/50.

🔗 СВЯЗЬ С ТЕХНИКАМИ

Negative тесты = основа техник: Граничные значения (BVA), Классы эквивалентности (ECP), Тестирование ошибок. Positive = Happy Path, Smoke Test.

🎯 ПРАВИЛО ПОЛНОГО ПОКРЫТИЯ

Для каждого тестируемого условия (поле, функция, API-метод) должны существовать ОБА типа тест-кейсов. Только положительные тесты создают ложную уверенность в качестве продукта. Только отрицательные — не проверяют основной функционал.

ПОЛНОЕ ПОКРЫТИЕ = Positive Test Cases + Negative Test Cases

1.4 НАВЫКИ QA-ИНЖЕНЕРА 2025–2026

Актуальная карта компетенций: Junior · Middle · Senior

ОСНОВЫ ТЕСТИРОВАНИЯ ПО

ТЕОРИЯ И ОСНОВЫ

Must Have — для всех уровней

- Виды тестирования: функциональное, регрессионное, интеграционное, Smoke, Sanity, приёмочное
- Техники тест-дизайна: классы эквивалентности, граничные значения (BVA), таблицы решений, попарное тестирование (Pairwise)
- Понимание SDLC / STLC, Agile (Scrum, Kanban)
- Написание тест-кейсов, тест-планов, чек-листов
- Баг-репорты: шаги воспроизведения, Severity, Priority
- Верификация и валидация, уровни тестирования

ИНСТРУМЕНТЫ JUNIOR

Must Have Junior / Junior+

- Баг-трекинг: Jira, YouTrack, Redmine
- Документация: Confluence, Notion
- Тест-менеджмент: Zephyr, Allure TestOps, TestRail
- API-тестирование: Postman, Swagger / OpenAPI
- SQL: SELECT, INSERT, UPDATE, DELETE, JOIN (INNER, LEFT, RIGHT), GROUP BY, WHERE, подзапросы
- UI: DevTools браузеры, BrowserStack, Figma (чтение макетов)
- Контроль версий: Git (базовые команды)
- Мобильное: Android Studio, Xcode, Charles Proxy / Proxyman

АВТОМАТИЗАЦИЯ

Must Have Middle — обязательно на рынке 2025–2026

- Языки: Python, JavaScript / TypeScript, Java
- UI-автотесты: Selenium, Playwright, Cypress
- API-автотесты: REST-assured, Requests (Python)
- BDD: Cucumber, Behave (Given / When / Then)
- TDD: принцип, JUnit, pytest
- CI/CD: Jenkins, GitLab CI, GitHub Actions, TeamCity
- Docker: поднять тестовое окружение
- Мониторинг: Kibana / Elastic, Grafana
- Linux: tail, grep, cat, ls, find

ADVANCED & AI (ТРЕНД 2026)

Middle+ и Senior

- Kafka, RabbitMQ, Redis — асинхронное тестирование
- Kubernetes — чтение логов подов
- Cloud: AWS / Azure / GCP (базовое)
- AI-инструменты: TestSigma, ACCELQ
- Data-driven тестирование: Python / SQL / Tableau
- Микросервисная архитектура

МАТРИЦА УРОВНЕЙ

✓ Must Have • Желательно — Не нужно

НАВЫК	JUN	MID	SEN
Теория тестирования	✓	✓	✓
SQL базовый	✓	✓	✓
SQL сложный (JOIN, GROUP BY)	•	✓	✓
REST API / Postman	✓	✓	✓
Git (базовые команды)	✓	✓	✓
DevTools / Figma	✓	✓	✓
Мобильное + снифферы	•	✓	✓
Kibana / Grafana / логи	•	✓	✓
Автоматизация Selenium/Playwright	•	✓	✓
Python / JS / Java	•	✓	✓
CI/CD Jenkins / GitLab CI	—	•	✓
Docker	—	•	✓
Kafka / RabbitMQ	—	•	✓
Kubernetes / Cloud	—	—	•
AI-инструменты тестирования	•	•	✓

⚠ Рынок 2025–2026: Junior без SQL, API и Git практически не берут. Middle обязан владеть хотя бы одним фреймворком автоматизации.

SOFT SKILLS —
Must Have для всех

Аналитическое
мышление

Коммуникация

Правильные вопросы
к требованиям

Приоритизация

Работа
в команде

1.5 РАЗНИЦА: QA / QC / ТЕСТИРОВАНИЕ

Иерархия концепций · Детальное сравнение · Обязанности на проекте

ОСНОВЫ ТЕСТИРОВАНИЯ ПО

ИЕРАРХИЯ

Testing $\subset$ QC $\subset$ QA

QA

Quality Assurance

QC

Quality Control

Testing

Тестирование
(Testing)

Контроль качества продукта

Обеспечение качества процессов

QA — проактивный подход.
Предотвращает появление дефектов
путём улучшения процессов.

QC — реактивный подход.
Обнаруживает несоответствия
в готовом продукте или его части.

Testing — реактивный подход.
Непосредственное выполнение
проверок продукта.

ДЕТАЛЬНОЕ СРАВНЕНИЕ ПО 7 КРИТЕРИЯМ

КРИТЕРИЙ	QA	QC	Testing
Расшифровка	Quality Assurance	Quality Control	—
Суть	Обеспечение качества Процессы разработки	Контроль продукта Выполнение проверок	Контроль продукта Выполнение проверок
Тип подхода	Проактивный — предотвращает	Реактивный — находит	Реактивный — фиксирует
Когда работает	На всех этапах SDLC	После создания артефактов	На этапе выполнения тестов
Объект	Процессы разработки и тестирования	Продукт и итоги тестирования	Продукт как результат
Отношение к процессу	Выстраивает процессы	Подмножество QA	Подмножество QC
Меры	Превентивные	Корректирующие	Выявление несоответствий

ОБЯЗАННОСТИ НА ПРОЕКТЕ

● QA Engineer

- Планирование и оптимизация процессов разработки
- Работа с документацией и требованиями
- Коммуникация с заказчиком
- Оценка рисков проекта
- Внедрение и актуализация методик тестирования
- Анализ работы тестировщиков и QC-специалистов
- Предотвращение появления дефектов

● QC Specialist

- Анализ результатов тестирования
- Предоставление информации о текущем уровне качества
- Работа с билдами в процессе разработки
- Проработка тест-кейсов и их результатов
- Составление отчётов о качестве

● Test Engineer

- Составление тест-кейсов и чек-листов
- Выполнение тест-кейсов согласно тест-плану
- Локализация дефектов
- Составление баг-репортов
- Взаимодействие с разработчиками при верификации исправлений
- Фокус непосредственно на тестировании



ВАЖНО НА ПРАКТИКЕ:

В небольших командах все три роли нередко совмещает один специалист с тайтлом "QA Engineer" или "Tester". Чёткое разграничение QA / QC / Testing — удел зрелых организаций и крупных проектов.



ЗАЧЕМ ЗНАТЬ ПРИНЦИПЫ

7 принципов — это фундамент профессионального мышления QA-специалиста, закреплённый в стандарте ISTQB CTFL 4.0. Они объясняют природу тестирования и предотвращают типичные ошибки новичков.

ISTQB CTFL 4.0

1 Тестирование показывает наличие дефектов

Testing shows presence of defects

Тестирование доказывает НАЛИЧИЕ дефектов, но НИКОГДА не может доказать их полное отсутствие. Прошедшие тесты ≠ баги отсутствуют.

✦ ПРИМЕР: 500 тест-кейсов пройдено. Завтра пользователь нашёл баг в сценарии, который не был протестирован.

ОШИБКА: "Все тесты прошли — продукт готов!"

2 Исчерпывающее тестирование невозможно

Exhaustive testing is impossible

Охватить все комбинации входных данных, условий и последовательностей действий физически невозможно. Нужна приоритизация по риску.

✦ ПРИМЕР: Поле "возраст" (0–150) × поле "имя" (любая строка) × 5 браузеров × 3 ОС = миллиарды комбинаций. Тестируем только критичные.

РЕШЕНИЕ: Risk-based testing · BVA · Pairwise

3 Раннее тестирование экономит время

Early testing saves time and money

Дефект на стадии требований исправляется в 100–1000 раз дешевле, чем в продакшне. Shift-Left = тестировщик участвует с первого дня.

✦ ПРИМЕР: QA замечает противоречие в User Story до написания кода — экономия 2 дней разработки. Vs. тот же баг в продакшне — 2 недели + деньги.

ПРИНЦИП: Shift-Left Testing

4 Скопление дефектов

Defect clustering (Pareto principle)

80% дефектов концентрируются в 20% модулей. Дефекты не распределены равномерно — они кластеризуются в "проблемных" местах кода.

✦ ПРИМЕР: На проекте e-commerce 70% багов — в модуле корзины и оплаты. QA уделяет им повышенное внимание при регрессии.

ВЫВОД: Усиливай тестирование рискованных модулей

5 Парадокс пестицида

Pesticide paradox

Одни и те же тесты, повторяемые снова и снова, теряют способность находить новые дефекты. Тест-сьют необходимо регулярно обновлять и дополнять.

✦ ПРИМЕР: Регрессионный набор не обновлялся 6 месяцев. Новая фича внесла баги в старый модуль, но ни один старый тест это не поймал. Нужны новые тест-кейсы!

ДЕЙСТВИЕ: Регулярно ревюить и пополнять тест-сьют.

6 Тестирование зависит от контекста

Testing is context-dependent

Нет универсального подхода. Тестирование медицинского ПО (ISO 62304) ≠ тестирование мобильной игры. Методы, глубина и стандарты определяются контекстом.

✦ ПРИМЕР: ПО для управления самолётом — требует формальные методы, строгая документация, многоуровневая верификация. Игровое приложение — экспресс-тесты, UX-фокус, A/B.

ПРИНЦИП: One size does NOT fit all в тестировании.

7 Заблуждение об отсутствии ошибок

Absence-of-errors fallacy

Нахождение и исправление дефектов бесполезно, если система не отвечает потребностям пользователей или не пригодна для использования по назначению.

✦ ПРИМЕР: Система бухгалтерского учёта без единого бага — но интерфейс настолько сложен, что бухгалтеры отказываются ей пользоваться. Качество ≠ только отсутствие багов.

КЛЮЧ: Validation важна не меньше Verification.

ТИПИЧНЫЕ ЗАБЛУЖДЕНИЯ НОВИЧКОВ

✗ Заблуждение

✓ Что на самом деле (принцип)

"Если все тесты прошли — продукт без багов"

Тестирование лишь снижает риск. Принцип №1.

"Нам нужно протестировать всё!"

Исчерпывающее тестирование невозможно. Принцип №2.

"Старые тесты защитят нас от новых багов"

Тест-сьют нужно обновлять. Принцип №5 — парадокс пестицида.

"Нет багов = качественный продукт"

Продукт должен решать задачу пользователя. Принцип №7.

БЛОК 2 из 7

Ошибка →
Дефект → Отказ

Факт. результат
vs Ожид. результат

Тест-план • Тест-кейс

Критерии
входа / выхода

Верификация
vs Валидация

Статическое
vs Динамическое

Баг-репорт • Метрики

баг

отказ

покрытие

Серьёзность
vs Приоритет

Жизненный цикл
дефекта

БЛОК 2

БАЗОВЫЕ ТЕРМИНЫ QA РАСШИРЕННЫЙ ГЛОССАРИЙ

Ошибка • Дефект • Серьёзность • Верификация • Баг-репорт • Метрики

2

СОДЕРЖАНИЕ БЛОКА 2

- | | | | |
|----------|---|--|---|
| 2.1 | Цепочка: Ошибка → Дефект → Отказ | 5 терминов • цепочка с примерами • Аномалия • Прерывание | ● |
| 2.2 | Факт. результат vs Ожид. результат | Фактический результат ≠ Ожидаемый результат = дефект • применение | ● |
| 2.3 | Терминология процесса тестирования | Документы (14 терминов) • Процессы (8 терминов) • полный глоссарий | ● |
| 2.4 | Критерии управления тестированием | Критерии входа/выхода • Приостановка • Возобновление • Определение готовности | ● |
| 2.5 | Верификация vs. Валидация | Верификация (Verification) vs Валидация (Validation) • 5 критериев сравнения | ● |
| 2.6 | Статическое vs. Динамическое тестирование | Методы • объекты • этапы применения • что находит каждый вид | ● |
| 2.7 | Серьёзность vs. Приоритет | Шкала C1–C5 • матрица серьёзность x приоритет • примеры из практики | ● |
| 2.8 | Жизненный цикл дефекта | 9 статусов • схема переходов • Переоткрыт • Отклонён • правила работы | ● |
| 2.9-2.10 | Баг-репорт и Метрики тестирования | 11 полей баг-репорта • 9 метрик с формулами • Эффективность обнаружения дефектов | ● |

"Знать термины — значит говорить на одном языке с командой."

— Принцип профессионального QA

Ошибка → Дефект → Отказ · Фактический vs Ожидаемый результат

ЦЕПочКА ВОЗНИКНОВЕНИЯ ДЕФЕКТА (ISTQB)

Понимание этой цепочки – базис профессии QA



НЕ КАЖДАЯ ОШИБКА ВЫЗЫВАЕТ ОТКАЗ:

Дефект может годами существовать в неиспользуемой ветке кода.
Отказ может быть вызван внешними условиями (радиация, сбой питания) без дефекта.

ПОЛНЫЙ СЛОВАРЬ ТЕРМИНОВ §2.1

ТЕРМИН	ОПРЕДЕЛЕНИЕ	ПРИМЕР
Ошибка (Error / Mistake)	Действие человека, приводящее к неверному результату. Существует в момент совершения действия.	Разработчик написал > вместо >=
Дефект (Defect / Bug / Fault)	Изъян в компоненте или системе, способный вызвать отказ при выполнении. Фиксируется в коде или документе.	Неверное условие в функции проверки скидки
Отказ (Failure)	Отклонение фактического поведения системы от ожидаемого, наблюдаемое при выполнении.	Приложение выдаёт неверный результат расчёта
Аномалия / Инцидент (Anomaly / Incident)	Любое состояние, отклоняющееся от ожиданий, стандартов или требований. Шире, чем отказ.	Кнопка реагирует не так, как описано в требованиях
Прерывание (Interruption)	Приостановка процесса, вызванная внешним событием, не связанным с дефектом в коде.	Зависание приложения из-за нехватки оперативной памяти

2.2 ФАКТИЧЕСКИЙ РЕЗУЛЬТАТ vs ОЖИДАЕМЫЙ РЕЗУЛЬТАТ

ОЖИДАЕМЫЙ РЕЗУЛЬТАТ
(Expected Result, ER)

Поведение, предсказанное спецификацией или другим авторитетным источником при заданных условиях. Определяется ДО выполнения теста.

ИСТОЧНИКИ:

- Требования (SRS, User Story)
- Дизайн-макеты (Figma, Zeplin)
- Предыдущая версия системы
- Экспертное суждение / стандарт

ФАКТИЧЕСКИЙ РЕЗУЛЬТАТ
(Actual Result, AR)

Поведение, реально наблюдаемое при выполнении теста на конкретной версии продукта. Фиксируется ПОСЛЕ выполнения.

КОГДА ФИКСИРОВАТЬ:

- При любом отклонении от ожидаемого результата
- Даже если отклонение кажется незначительным
- С точными данными: значения, сообщения, коды ошибок
- Со скриншотом или видеозаписью



КЛЮЧЕВОЕ ПРАВИЛО: Дефект фиксируется тогда и только тогда, когда

ФР ≠ ОР

Фактический результат не совпадает с ожидаемым → тест упал (Fail) → создаётся баг-репорт.



ДОКУМЕНТЫ И АРТЕФАКТЫ ТЕСТИРОВАНИЯ

(Test Documentation & Artifacts — ISTQB CTFL 4.0)

ТЕРМИН	ОПРЕДЕЛЕНИЕ
Тест-политика (Test Policy)	Высокоуровневый документ организации: зачем и как проводится тестирование. Создаётся один раз для всей компании.
Тест-стратегия (Test Strategy)	Описание уровней тестирования и подходов в рамках программы или организации.
Тест-план (Test Plan)	Документ с целями тестирования, ресурсами, расписанием и способами их достижения.
Мастер тест-план (Master Test Plan)	Тест-план, координирующий несколько уровней или видов тестирования одновременно.
Основа тестирования (Test Basis)	Совокупность знаний для анализа и дизайна тестов: требования, код, дизайн, стандарты.
Объект тестирования (Test Object)	Рабочий продукт, подлежащий тестированию: компонент, модуль, API, документ.
Область тестирования (Test Scope)	Описание тестируемого объекта и перечень его проверяемых функций и характеристик.
Тест-кейс (Test Case)	Набор входных значений, предусловий, ожидаемых результатов и постусловий для достижения конкретной тест-цели.
Тест-сюит (Test Suite)	Набор тест-кейсов или тест-скриптов для выполнения в рамках одного тест-рана.
Тест-скрипт (Test Script)	Последовательность детальных инструкций для выполнения теста (ручных или автоматизированных).
Тест-ран (Test Run)	Конкретный запуск тест-сюита на конкретной версии продукта с фиксацией результатов.
Тест-лог (Test Log)	Хронологическая запись деталей выполнения тестов: время, статус, фактические результаты.
Чек-лист (Checklist)	Список проверяемых пунктов или атрибутов продукта без детальных шагов воспроизведения.
Отчёт о дефекте / Баг-репорт (Defect Report)	Документ, фиксирующий факт, природу, воспроизводимость и текущий статус дефекта.



ПРОЦЕССЫ ТЕСТИРОВАНИЯ

(Testing Processes — ISTQB CTFL 4.0)

Анализ тестирования (Test Analysis)	Определение тестовых условий путём анализа основы тестирования. Отвечает: "ЧТО тестировать?"
Проектирование тестов (Test Design)	Разработка и спецификация тест-кейсов из тест-условий. Отвечает: "КАК тестировать?"
Реализация тестов (Test Implementation)	Подготовка тестового окружения, тест-данных и тест-кейсов для выполнения.
Выполнение тестов (Test Execution)	Запуск тестов и получение фактических результатов с фиксацией в тест-логе.
Завершение тестирования (Test Completion)	Передача тестового обеспечения для дальнейшего использования, подготовка итогового отчёта.
Мониторинг тестирования (Test Monitoring)	Проверка текущего статуса тестирования, выявление отклонений от тест-плана.
Управление тестированием (Test Control)	Применение корректирующих действий при обнаружении отклонений от плана.
Отладка (Debugging) ⚠	Поиск, анализ и устранение ПРИЧИН отказов. ВНИМАНИЕ: выполняет разработчик, не тестировщик!



ИЕРАРХИЯ: Тест-политика ⊃ Тест-стратегия ⊃ Мастер тест-план ⊃ Тест-план ⊃ Тест-сюит ⊃ Тест-кейс

СВОДНАЯ ТАБЛИЦА 7 КРИТЕРИЕВ

Критерий	Когда применяется	Что означает
Критерии входа (Entry Criteria)	Перед началом	Условия, при которых тестирование разрешено начать
Критерии выхода (Exit Criteria)	По завершении	Условия, при которых тестирование считается выполненным
Критерии приостановки (Suspension Criteria)	В процессе	Условия для временной приостановки тест-активностей
Критерии возобновления (Resumption Criteria)	После паузы	Условия для возобновления приостановленного тестирования
Критерии приёмки (Acceptance Criteria)	Перед приёмкой	Критерии, которым должен соответствовать продукт для принятия заказчиком
Определение готовности (Definition of Done, DoD)	Agile	Согласованный набор условий завершенности задачи или пользовательской истории
Критерии прохождения (Pass / Fail Criteria)	На тест-кейс	Правила определения: тест пройден или упал для конкретного тест-кейса

● КРИТЕРИИ ВХОДА (Entry Criteria)

Условия, при выполнении которых разрешено начинать тест-активности.

- Сборка передана в тестовое окружение
- Дымовое тестирование пройдено успешно
- Тест-кейсы разработаны и прошли ревью
- Тестовые данные подготовлены

🚩 Тестирование спринта начинается после успешного дымового теста $\geq 95\%$ пройдено.

● КРИТЕРИИ ВЫХОДА (Exit Criteria)

Условия, при выполнении которых тест-активности считаются завершенными.

- Все тест-кейсы выполнены
- Процент прохождения $\geq$ установленного порога
- Все критические и блокирующие дефекты закрыты
- Регрессионное тестирование пройдено

🚩 Релиз: 100% тест-кейсов выполнено, 97% пройдено, 0 открытых блокеров.

II КРИТЕРИИ ПРИОСТАНОВКИ (Suspension Criteria)

Условия, при которых тест-активности временно останавливаются.

- Тестовое окружение недоступно
- Блокирующий дефект блокирует $>30\%$ тест-кейсов
- Тестовые данные повреждены
- Изменения требований требуют пересмотра тестов

При паузе фиксируется: причина, статус, дата приостановки.

▶ КРИТЕРИИ ВОЗОБНОВЛЕНИЯ (Resumption Criteria)

Условия для продолжения приостановленного тестирования.

- Причина приостановки устранена
- Окружение восстановлено и проверено
- Блокирующий дефект исправлен и верифицирован
- Команда разработки подтвердила готовность

Критерии возобновления = зеркало критериев приостановки.



ОПРЕДЕЛЕНИЕ ГОТОВНОСТИ (Definition of Done, DoD) В КОНТЕКСТЕ AGILE

Согласованный командой список условий, при выполнении которых задача (пользовательская история, фича) считается полностью завершенной. В контексте QA в DoD обычно входит: тест-кейсы написаны и выполнены, дефекты закрыты, регрессия пройдена, документация обновлена.

Severity vs. Priority · Классическая ошибка новичков — путаница этих понятий

КЛЮЧЕВЫЕ ОТЛИЧИЯ

Критерий	Серьёзность (Severity)	Приоритет (Priority)
Что описывает	Степень влияния дефекта на ФУНКЦИОНАЛЬНОСТЬ системы	Степень важности исправления дефекта для БИЗНЕСА
Кто определяет	QA / тестировщик — на основе технического влияния	Менеджер продукта / заказчик — на основе бизнес-ценности
Меняется ли?	Редко — технический факт	Часто — зависит от спринта и релиза

💡 **КЛАССИЧЕСКИЙ ПРИМЕР:** Опечатка в логотипе на главной странице.
Серьёзность: НИЗКАЯ — не ломает функциональность системы.
Приоритет: ВЫСОКИЙ — критично для репутации бренда, нужно исправить до релиза.

ШКАЛА СЕРЬЁЗНОСТИ — C1 ДО C5

C1	БЛОКЕР (Blocker) Тестирование невозможно продолжить. Система не запускается или полностью неработоспособна.	Сбой приложения при запуске
C2	КРИТИЧЕСКИЙ (Critical) Основной функционал сломан, обходного пути нет. Ключевые сценарии недоступны.	Невозможно оформить заказ
C3	ЗНАЧИТЕЛЬНЫЙ (Major) Основной функционал сломан, но есть обходной путь. Работа ограничена.	Оплата картой не работает, оплата через PayPal работает
C4	НЕЗНАЧИТЕЛЬНЫЙ (Minor) Частичное нарушение функциональности с малым влиянием на основные сценарии.	Фильтр работает, но неверно сортирует результаты
C5	ТРИВИАЛЬНЫЙ (Trivial) Косметический дефект. Нет влияния на функциональность. Только внешний вид.	Опечатка в тексте всплывающей подсказки

МАТРИЦА СЕРЬЁЗНОСТЬ × ПРИОРИТЕТ

	Приоритет: ВЫСОКИЙ 🚨	Приоритет: НИЗКИЙ 🐢
Серьёзность: ВЫСОКАЯ ↑	ИСПРАВИТЬ НЕМЕДЛЕННО Критический дефект в критичной функции. Блокирует релиз. Исправление начинается сейчас. <i>Сбой при оплате в Чёрную пятницу</i>	ИСПРАВИТЬ В БЛИЖАЙШЕМ СПРИНТЕ Серьёзный дефект, но низкий бизнес-приоритет. Планируется, не горит. <i>Критический дефект в редко используемом отчёте</i>
Серьёзность: НИЗКАЯ ↓	ИСПРАВИТЬ ДО РЕЛИЗА Заметный дефект: пользователь видит, некритично технически, важно для бренда. <i>Опечатка в логотипе на главной</i>	ИСПРАВИТЬ КОГДА-НИБУДЬ Минимальное влияние. Откладывается на усмотрение команды. <i>Лишний пробел в тексте справки</i>

2.9 КЛЮЧЕВЫЕ ПОЛЯ БАГ-РЕПОРТА

ПОЛЕ (FIELD)	ЧТО УКАЗЫВАТЬ
1. Заголовок (Summary)	Краткое описание: Что? Где? Когда? Не длиннее одной строки. Без воды.
2. Описание (Description)	Подробное описание дефекта, контекст, дополнительный контекст.
3. Шаги воспроизведения (Steps to Reproduce)	Нумерованный список точных действий для воспроизведения дефекта.
4. Фактический результат (Actual Result)	Что происходит по факту — точные данные, сообщения, коды ошибок.
5. Ожидаемый результат (Expected Result)	Что должно было произойти согласно требованиям или спецификации.
6. Серьёзность (Severity)	Степень влияния на функциональность: C1 Блокер / C2 Критический / C3 Значительный / C4 Незначительный / C5 Тривиальный
7. Приоритет (Priority)	Бизнес-важность исправления: Высокий / Средний / Низкий
8. Окружение (Environment)	ОС, браузер и версия, версия приложения, устройство, разрешение экрана.
9. Вложения (Attachments)	Скриншот, видеозапись, лог-файлы, HAR-файл сетевых запросов.
10. Проект / Компонент (Component)	К какому модулю или разделу системы относится дефект.
11. Воспроизводимость (Reproducibility)	Всегда / Иногда / Случайно / Не удаётся воспроизвести

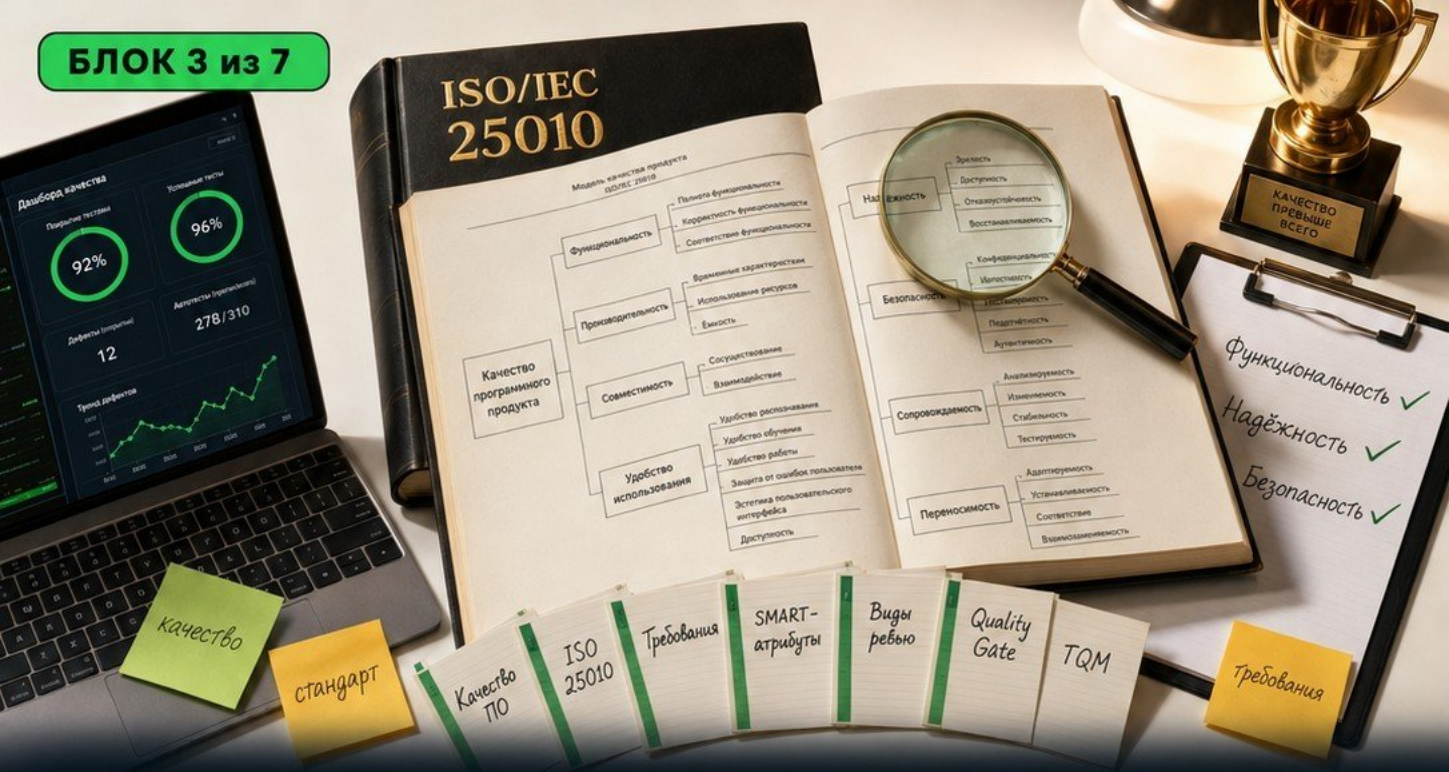
● Обязательные поля ● Важные поля ● Дополнительные поля

2.10 МЕТРИКИ ТЕСТИРОВАНИЯ

МЕТРИКА	ФОРМУЛА / ОПИСАНИЕ
1. Покрытие тестами (Test Coverage)	$(\text{Число покрытых условий} \div \text{Всего условий}) \times 100\%$
2. Плотность дефектов (Defect Density)	$(\text{Число дефектов} \div \text{Размер продукта (строки кода, функциональные точки)})$
3. Процент прохождения (Pass Rate)	$(\text{Число пройденных тестов} \div \text{Всего выполненных тестов}) \times 100\%$
4. Процент провалов (Fail Rate)	$(\text{Число упавших тестов} \div \text{Всего выполненных тестов}) \times 100\%$
5. Ускользнувший дефект (Escaped Defect)	Дефект, не обнаруженный на предназначенном этапе тестирования и попавший в продакшн
6. Эффективность обнаружения дефектов (DDP)	$(\text{Дефекты на этапе} \div (\text{Те же дефекты} + \text{найденные позже})) \times 100\%$
7. Эффективность устранения дефектов (DRE)	$(\text{Дефекты до релиза} \div (\text{Дефекты до релиза} + \text{в продакшне})) \times 100\%$
8. Среднее время до отказа (MTTF)	(Mean Time To Failure) — среднее время работы системы до следующего отказа
9. Среднее время восстановления (MTTR)	(Mean Time To Repair) — среднее время устранения отказа и восстановления системы



КЛЮЧЕВАЯ МЕТРИКА КОМАНДЫ: Эффективность обнаружения дефектов (DDP) — показывает, насколько хорошо тестирование ловит дефекты ДО продакшна.
Цель: DDP ≥ 90%.



БЛОК 3

КАЧЕСТВО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

СТАНДАРТЫ • ТРЕБОВАНИЯ • РЕВЬЮ

ISO/IEC 25010 • ISTQB CTFL 4.0 • IREB • Гарвин • SMART-требования

3

СОДЕРЖАНИЕ БЛОКА 3

- 3.1** **Что такое качество ПО?**
Определения IREB и ISO • 5 взглядов Гарвина • примеры
- 3.2A** **Модель ISO/IEC 25010 — часть I**
Характеристики 1–4: Функциональность • Производительность • Совместимость • Удобство
- 3.2B** **Модель ISO/IEC 25010 — часть II**
Характеристики 5–8: Надёжность • Безопасность • Сопровождаемость • Переносимость
- 3.3** **Функциональные vs Нефункциональные требования**
Иерархия требований • сравнение • типы NFR • пирамида
- 3.4** **Атрибуты качественного требования**
10 SMART-атрибутов • слова-маркеры плохих требований • практика QA
- 3.5** **Виды ревью (Статическое тестирование)**
4 вида ревью • сравнение формальности • роли при Инспекции
- 3.6** **Дополнительные понятия о качестве**
Quality Gate • Quality Debt • TQM • Матрица трассируемости • Root Cause Analysis

Качество — это не действие. Это привычка.

— Аристотель (о качестве в разработке ПО)

Определения IREB и ISO/IEC 25010 · 5 взглядов на качество по Гарвину



ОПРЕДЕЛЕНИЯ КАЧЕСТВА

IREB

Требований и разработки

Степень, в которой рабочий продукт удовлетворяет заявленным и подразумеваемым потребностям заинтересованных сторон.

Ключевое слово:

ISO/IEC 25010

Систем и программного обеспечения

Степень, в которой система или программное обеспечение обладает набором присущих характеристик, которые удовлетворяют требованиям заинтересованных сторон.

Ключевое слово:

5 ВЗГЛЯДОВ НА КАЧЕСТВО ПО ГАРВИНУ

№	Взгляд (подход)	Ключевой вопрос	Фокус	Критерии оценки	Примеры показателей
1	Субъективный (Transcendent)	Это хорошее ПО?	Восприятие качества в целом	Эстетика, престиж, имидж, «эффект вау»	Бренд, репутация, дизайн, эмоциональное впечатление
2	Продуктовый (Product-based)	Что в нём есть?	Характеристики и функциональность	Набор функций, надёжность, производительность	Покрывание функций, дефекты, баги на тысячу строк кода
3	Пользовательский (User-based)	Подходит ли оно мне?	Удовлетворение пользователя	Полезность, удобство, простота использования	Удовлетворённость (NPS), удобство (SUS), удержание
4	Производственный (Manufacturing-based)	Как хорошо оно сделано?	Соответствие процессам и стандартам	Соответствие спецификациям, качество процессов	Дефекты, соответствие стандартам, тестовое покрытие
5	Ценностный (Value-based)	Какую ценность оно даёт?	Ценность и выгода для бизнеса	ROI, снижение рисков, повышение эффективности	ROI, экономия затрат, Uptime, снижение инцидентов

8 ХАРАКТЕРИСТИК КАЧЕСТВА ПО (ISO/IEC 25010)

СТРАНИЦА 1 ИЗ 2 — Характеристики 1–4

1

**Функциональная пригодность**
(Functional Suitability)

2

**Производительность**
(Performance Efficiency)

3

**Совместимость**
(Compatibility)

4

**Удобство использования**
(Usability)

Текущая страница: 1–4

ПОДХАРАКТЕРИСТИКИ — ДЕТАЛЬНАЯ КАРТА

**1. ФУНКЦИОНАЛЬНАЯ ПРИГОДНОСТЬ** (Functional Suitability)

Наличие и корректность функций, соответствующих потребностям

Функциональная полнота (Completeness)	Все нужные функции присутствуют в системе. Нет пропущенных сценариев.	Все пункты меню из ТЗ реализованы
Функциональная корректность (Correctness)	Функции дают правильные результаты при правильных входных данных.	Расчёт скидки 10% = точный результат
Функциональная целесообразность (Appropriateness)	Функции помогают решить задачи пользователя в нужном контексте.	Функция экспорта в CSV актуальна для аналитика

**2. ПРОИЗВОДИТЕЛЬНОСТЬ** (Performance Efficiency)

Соответствие временному поведению и использованию ресурсов

Временное поведение (Time Behaviour)	Время отклика, обработки и пропускная способность при выполнении функций.	Страница загружается < 2 сек при 1000 пользователей
Использование ресурсов (Resource Utilisation)	Количество потребляемых ресурсов: CPU, память, диск, сеть.	CPU не превышает 80% при пиковой нагрузке
Ёмкость (Capacity)	Максимальные пределы параметров системы: транзакции, пользователи, данные.	Система поддерживает 10 000 одновременных сессий

**3. СОВМЕСТИМОСТЬ** (Compatibility)

Способность сосуществовать и взаимодействовать с другими системами

Сосуществование (Co-existence)	Продукт может работать в одной среде с другими продуктами, не мешая им.	Расширение браузера не конфликтует с антивирусом
Взаимодействие (Interoperability)	Система может обмениваться данными и работать совместно с другими системами.	API возвращает данные в формате, понятном CRM-системе

**4. УДОБСТВО ИСПОЛЬЗОВАНИЯ** (Usability)

Степень, в которой продукт используется с эффективностью и удовлетворённостью

Распознаваемость (Appropriateness Recognisability)	Пользователь понимает, подходит ли продукт для его задачи
Обучаемость (Learnability)	Простота освоения продукта новыми пользователями
Управляемость (Operability)	Простота управления и использования продукта
Защита от ошибок пользователя (User Error Protection)	Система предотвращает неверные действия пользователя
Эстетика интерфейса (UI Aesthetics)	Приятный и удовлетворяющий визуальный интерфейс
Доступность (Accessibility)	Продукт доступен людям с ограниченными возможностями

8 ХАРАКТЕРИСТИК КАЧЕСТВА ПО (ISO/IEC 25010)

СТРАНИЦА 2 ИЗ 2 — Характеристики 5–8

5. Надёжность
(Reliability)

5

6. Безопасность
(Security)

6

7. Сопровождаемость
(Maintainability)

7

8. Переносимость
(Portability)

8



Характеристики 5–8 — завершение модели



5. НАДЁЖНОСТЬ (Reliability)

Способность системы выполнять функции при заданных условиях

- Безотказность (Maturity)** Частота отказов при нормальной эксплуатации
- Доступность (Availability)** Процент времени работоспособности системы (uptime) | Цель: 99.9% uptime
- Отказоустойчивость (Fault Tolerance)** Работа системы несмотря на сбой компонентов
- Восстанавливаемость (Recoverability)** Скорость восстановления после отказа — метрика MTTR



6. БЕЗОПАСНОСТЬ (Security)

Защита информации и данных от несанкционированного доступа

- Конфиденциальность (Confidentiality)** Данные доступны только авторизованным пользователям
- Целостность (Integrity)** Защита от несанкционированного изменения данных
- Неотказуемость (Non-repudiation)** Невозможность отрицать факт выполненного действия
- Подотчётность (Accountability)** Все действия пользователя журналируются
- Аутентичность (Authenticity)** Подтверждение личности и подлинности данных



7. СОПРОВОЖДАЕМОСТЬ (Maintainability)

- Модульность (Modularity)** Система состоит из независимых компонентов
- Повторное использование (Reusability)** Компоненты можно использовать в других системах
- Анализируемость (Analysability)** Простота оценки влияния изменений и поиска причин отказов
- Модифицируемость (Modifiability)** Лёгкость внесения изменений без деградации
- Тестируемость (Testability)** Простота создания тестов и критериев проверки



8. ПЕРЕНОСИМОСТЬ (Portability)

- Адаптируемость (Adaptability)** Адаптация к разным средам без дополнительных действий
- Устанавливаемость (Installability)** Простота установки и удаления в целевой среде
- Заменяемость (Replaceability)** Возможность замены другим продуктом аналогичного назначения

ЧТО ТЕСТИРУЕМ ПО КАЖДОЙ ХАРАКТЕРИСТИКЕ

Характеристика	Вид тестирования	Инструменты
Функциональная пригодность	Функциональное тестирование	Тест-кейсы, чек-листы, API-тесты
Производительность	Нагрузочное, стресс-тестирование, объёмное	JMeter, Gatling, k6
Совместимость	Тестирование совместимости, интеграционное	BrowserStack, кросс-браузерные тесты
Удобство использования	Юзабилити-тестирование, тестирование доступности	WAVE, axe, ручное
Надёжность	Тестирование надёжности, Failover, восстановление	Chaos Engineering, мониторинг
Безопасность	Тестирование безопасности, пентест	OWASP, Burp Suite
Сопровождаемость	Статический анализ, ревью кода	SonarQube, PMD
Переносимость	Инсталляционное, кросс-платформенное	Ручное, эмуляторы

ИЕРАРХИЯ ТРЕБОВАНИЙ

Каждый уровень конкретизирует предыдущий

БИЗНЕС-ТРЕБОВАНИЯ

(Business Requirements)

Что нужно бизнесу?



конкретизирует

ПОЛЬЗОВАТЕЛЬСКИЕ ТРЕБОВАНИЯ

(User Requirements)

Что должен уметь делать пользователь?



конкретизирует

СИСТЕМНЫЕ ТРЕБОВАНИЯ

(System Requirements)

Функциональные

(Functional)

ЧТО делает

Нефункциональные

(Non-Functional)

КАК ХОРОШО делает

Ограничения

(Constraints)

В каких рамках

СРАВНЕНИЕ ФУНКЦИОНАЛЬНЫХ И НЕФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

Критерий	Функциональные (FR)	Нефункциональные (NFR)
Вопрос	Что система делает?	Как хорошо делает?
Описывает	Поведение, функции, возможности системы	Атрибуты качества и ограничения выполнения
При нарушении	Система работает НЕПРАВИЛЬНО	Система работает, но ПЛОХО
Пример	Пользователь может войти по логину и паролю	Страница авторизации загружается менее чем за 2 секунды

ТИПИЧНЫЕ ВИДЫ НЕФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

Производительность
(Performance)

Время загрузки страницы < 2 сек
при 1000 пользователей
одновременно

Безопасность
(Security)

Пароль должен хэшироваться
алгоритмом bcrypt.
HTTPS обязателен.

Масштабируемость
(Scalability)

Система должна поддерживать
рост нагрузки в 10 раз
без деградации.

Доступность
(Accessibility - WCAG 2.1 AA)

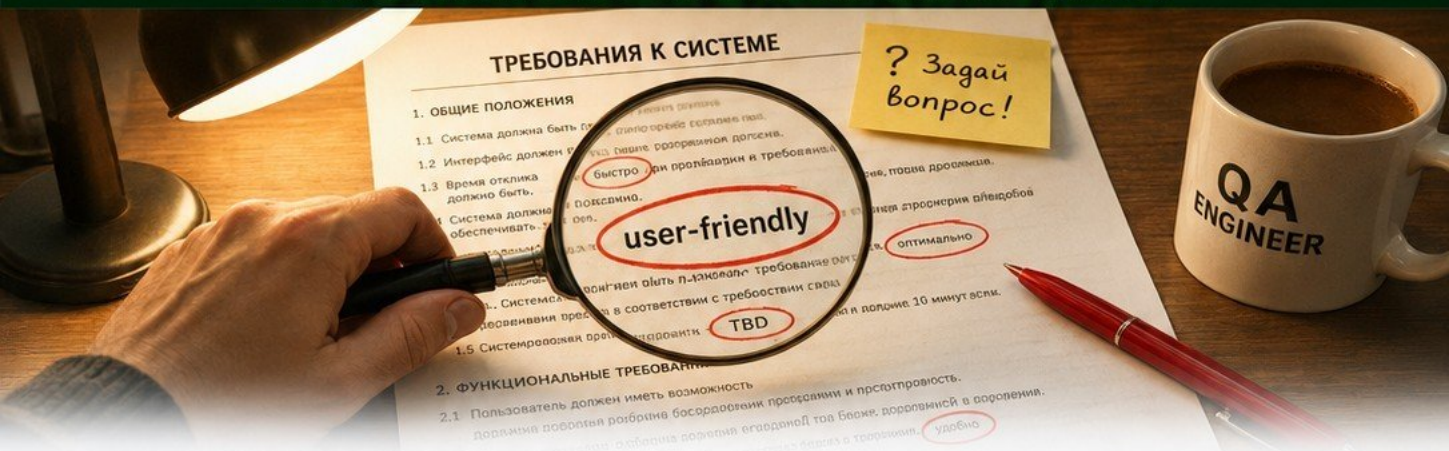
Интерфейс должен быть
доступен для пользователей
с нарушениями зрения.

Надёжность
(Reliability / Uptime)

Доступность системы: 99.9% -
не более 8.7 часов
простоя в год.

Сопровождаемость
(Maintainability)

Покрываемость кода тестами ≥ 80%.
Технический долг < 5%
от времени разработки.



10 АТТРИБУТОВ КАЧЕСТВЕННОГО ТРЕБОВАНИЯ

Если хотя бы один атрибут нарушен — это дефект документации. Фиксируй до начала разработки!

Атрибут	Что означает	Признак нарушения
1 Полнота (Completeness)	Всё необходимое для реализации и тестирования присутствует	Нет ожидаемого результата или формата данных
2 Атомарность (Atomicity)	Одно требование = одна потребность бизнеса	"Система должна сохранять И отправлять..." — это два требования
3 Непротиворечивость (Consistency)	Не конфликтует с другими требованиями документа	Разные разделы описывают одно поле по-разному
4 Однозначность (Unambiguousness)	Только одна возможная интерпретация	Слова: "быстро", "удобно", "оптимально", TBD
5 Осуществимость (Feasibility)	Реализуемо в рамках бюджета, сроков, технологий	"Система должна отвечать мгновенно" без конкретики
6 Трассируемость (Traceability)	Связано с источником, отслеживается до тест-кейса	Нет ID требования, невозможно найти первоисточник
7 Корректность (Correctness)	Точно описывает нужную функциональность	Описывает не то, что действительно нужно бизнесу
8 Верифицируемость (Verifiability)	Можно объективно проверить выполнение	Нельзя написать тест-кейс → требование не верифицируемо
9 Модифицируемость (Modifiability)	Изменение не ломает структуру документа	Одно и то же требование дублируется в нескольких местах
10 Приоритизированность (Prioritization)	Требованиям присвоены приоритеты Must/Should/Could/Won't	Все требования обозначены как "важные"



СЛОВА-МАРКЕРЫ ПЛОХИХ ТРЕБОВАНИЙ

Увидел одно из этих слов → требование неоднозначное → ЗАДАЙ УТОЧНЯЮЩИЙ ВОПРОС

improved **user-friendly** TBD robust as applicable
adequate easy fast simple normal flexible
efficient if possible but not limited to to be determined
timely as appropriate **optimal** minimize maximize state-of-the-art

Эти слова делают требование не верифицируемым

4 вида ревью · Инспекция (Inspection) · 5 ролей · Статический анализ



ШКАЛА ФОРМАЛЬНОСТИ РЕВЬЮ

Неформальное (Informal Review)	Обзор (Walkthrough)	Техническое ревью (Technical Review)	Инспекция (Inspection)
-----------------------------------	------------------------	---	---------------------------

← Меньше документации · Меньше ролей · Меньше дефектов → Больше документации · Больше ролей · Больше дефектов →

СРАВНЕНИЕ 4 ВИДОВ РЕВЬЮ (ISTQB CTFL 4.0)

Критерий	Неформальное	Обзор	Техническое ревью	Инспекция
Формальность	Минимальная	Низкая	Средняя	Максимальная
Кто ведёт	Любой	АВТОР документа	Модератор (не автор)	Обученный модератор
Роли	Не определены	Автор + ревьюеры	Коллеги-эксперты	Строго определены
Документация	✗ Не требуется	⚠ Опционально	✓ Требуется	✓✓ Обязательна
Цель	Быстрая проверка	Понимание + вопросы	Технический консенсус	Метрики дефектов
Применяется	Небольшие изменения, код	Требования, дизайн	Архитектура, дизайн	Критическая документация

5 РОЛЕЙ ПРИ ИНСПЕКЦИИ (Inspection — ISTQB)

Модератор (Moderator) Планирует и ведёт ревью, следит за регламентом	Автор (Author) Создал рецензируемый артефакт, отвечает на вопросы	Ревьюер (Reviewer / Inspector) Находит дефекты в артефакте до встречи	Секретарь (Scribe / Recorder) Фиксирует найденные дефекты и решения команды	Чтец (Reader) Зачитывает документ вслух во время встречи
---	--	--	--	---

ВИДЫ СТАТИЧЕСКОГО АНАЛИЗА (Static Analysis)

Ревью кода (Code Review) Коллеги проверяют код на стандарты, логику, уязвимости	Автоматический анализ (Static Analysis) Инструменты: SonarQube, PMD, ESLint без запуска	Анализ потока управления (Control Flow Analysis) Анализ последовательности выполнения операций
Анализ потока данных (Data Flow Analysis) Жизненный цикл переменных: определение → использование → удаление	Ревью требований (Requirements Review) Проверка на полноту, однозначность, непротиворечивость	Инспекция кода (Code Inspection) Формальный процесс с чеклистом, ролями и метриками



ГЛОССАРИЙ — 9 КЛЮЧЕВЫХ ПОНЯТИЙ

ТЕРМИН	ОПРЕДЕЛЕНИЕ
Контрольная точка качества (Quality Gate)	Контрольный порог - нельзя перейти к следующей фазе SDLC без выполнения заданных критериев качества. Аналог светофора для релиза.
Долг по качеству (Quality Debt)	Накопленные отложенные работы по обеспечению качества. Аналог технического долга (Technical Debt), но в области QA-процессов.
Культура качества (Quality Culture)	Система ценностей и установок организации, направленная на непрерывное улучшение качества всеми участниками процесса.
Всеобщее управление качеством (TQM)	Total Quality Management - общеорганизационный подход к управлению качеством. Качество как ответственность каждого, а не только QA.
Встроенное качество (Built-in Quality, SAgE)	Принцип Agile/SAgE: качество встраивается в каждый инкремент разработки, а не проверяется только в конце спринта или релиза.
Анализ первопричин (Root Cause Analysis, RCA)	Методика выявления коренной причины возникновения дефекта с целью предотвращения его повторного появления в будущем.
Предотвращение дефектов (Defect Prevention)	Проактивные меры по снижению вероятности появления дефектов: обучение, стандарты, ревью, автоматизация.
Изоляция дефектов по фазам (Phase Containment)	Политика: дефекты должны быть устранены на той же стадии SDLC, где они были внесены. Чем позже - тем дороже.
Матрица трассируемости (Traceability Matrix, RTM)	Таблица связей: требование ↔ тест-кейс. Позволяет оценить покрытие требований тестами и найти непокрытые зоны.

МАТРИЦА ТРАССИРУЕМОСТИ (Requirements Traceability Matrix, RTM) — ПРИМЕР

ТРЕБОВАНИЕ	ТК-001	ТК-002	ТК-003	ТК-004
ТРБ-001 · Вход по логину	✓	✓	—	—
ТРБ-002 · Сброс пароля	—	✓	✓	—
ТРБ-003 · Фильтр товаров	—	—	✓	✓
ТРБ-004 · Оплата картой	—	—	—	✓
ТРБ-005 · Экспорт в PDF	—	—	—	✗ НЕТ ТЕСТА

⚠ **Требование без тест-кейса = пробел в покрытии. Нужно создать тест!** ← ----- ↑

✓ — тест-кейс покрывает требование · ✗ — требование не покрыто · — не связаны

💡 **RTM позволяет QA-лиду ответить на вопрос: "Все ли требования проверены?" ДО начала регрессионного тестирования.**



БЛОК 3 ЗАВЕРШЁН · Следующий блок: Уровни и виды тестирования

ISO/IEC 25010 · Гарвин · SMART-требования · Ревью · Quality Gate - базис профессионального QA